

MICROSERVICE-BASED CAR RENTAL SYSTEM

ACADEMIC PROJECT DOCUMENTATION AND TECHNICAL REPORT

Mehmet Atakan Kılıç - 212010020100

Emre Haksever - 212010020101

PROJECT SUMMARY

This project is an end-to-end Car Rental System that practically demonstrates modern software architecture principles: **Microservices Architecture**, **Containerization (Docker)**, **Continuous Integration/Continuous Deployment (CI/CD - Jenkins)**, and **System Observability (Datadog & Prometheus)**.

The system consists of 3 independently scalable microservices, isolated databases, and DevOps automation tools. All technologies used in the project are industry standards and are designed in accordance with modern Cloud-Native application development standards.

CONTENTS

1. Introduction and Problem Statement
2. System Architecture and Design Principles
3. Detailed Analysis of Microservices and Data Models
 - 3.1. User Service
 - 3.2. Vehicle Service
 - 3.3. Rental Service
4. Inter-Microservice Communication Protocol
5. Infrastructure and DevOps Processes (Docker & Jenkins Pipeline)
6. Observability and Monitoring Stack (Datadog APM & Prometheus)
7. API Test Scenarios (Postman)
8. Academic Evaluation and Conclusion

1. INTRODUCTION AND PROBLEM STATEMENT

Traditional monolithic software architectures create disadvantages such as maintenance difficulties, single point of failure, technology dependency, and scaling challenges as the project grows.

In this project, the limitations of monolithic architecture are overcome; **Car Rental** business processes are divided into microservices that are independent of each other, focused on their own areas of responsibility (Single Responsibility), and manage their own databases (Database-per-Service).

Primary Objectives of the Project:

- Secure management of user authorization and profiles.
- Dynamic update and tracking of the vehicle inventory.

- Consistent execution of rental operations through synchronous HTTP API calls between services.
- Automated testing and deployment of the code via Jenkins.
- Centralized monitoring of system performance data and logs using Datadog and Prometheus.

2. SYSTEM ARCHITECTURE AND DESIGN PRINCIPLES

The architectural design of the system aims for the services to be loosely coupled. The interaction between the system components is based on routing JWT-authenticated requests from the client to the relevant services, while the services interact with their own isolated MongoDB databases (Database-per-Service).

Core Architectural Principles:

1. **Database-per-Service Pattern:** Each microservice can only access its own MongoDB database. user-service can only read/write to user_db, vehicle-service to vehicle_db, and rental-service to rental_db. Direct SQL/NoSQL queries from one service to another's database are blocked.
2. **Stateless Services:** Services are completely stateless. User session information is not kept on the server; it is instantly verified with a JSON Web Token (JWT) sent by the client. This allows the services to scale infinitely horizontally by placing a Load Balancer behind them.
3. **CI/CD Pipeline As Code:** Deployment steps are not manual but defined as code in the version control system (Jenkinsfile).

3. DETAILED ANALYSIS OF MICROSERVICES AND DATA MODELS

3.1. User Service

This service handles user registration, login, and authorization (Role-Based Access Control). It operates on Port: 3001.

- **Technology Stack:** Node.js, Express, Mongoose, JWT, BcryptJS.
- **Security:** User passwords are not saved as plain text in the database. They are hashed using the bcryptjs library with 10 salt rounds. Upon login, a signed JWT token containing the user's userId and role, valid for 1 hour, is generated.

API Endpoints:

- POST /api/auth/register : New user registration. By specifying the role parameter, an admin or standard user can be created.
- POST /api/auth/login : User login. Returns a JWT if successful.
- GET /api/auth/profile : Retrieves the profile information of the token owner (excluding password). Protected by authMiddleware.
- GET /api/auth/users : Lists all users in the system. Accessible only by users with role: admin.

3.2. Vehicle Service

This service keeps track of the vehicle inventory in the fleet and manages their availability status. It operates on Port: 3002.

- **Security Layers: * Admin Protection:** Adding a new vehicle can only be done with the JWT token of users with role: admin.
 - **Internal API Key Protection:** The authority to update a vehicle's availability status (isAvailable) during the rental process belongs only to internal inter-service communication. To prevent unauthorized external users from changing this status, an x-internal-api-key check is performed via HTTP Headers.

API Endpoints:

- POST /api/vehicles : Adds a new vehicle to the inventory (Admin & JWT protected).
- GET /api/vehicles/available : Lists the vehicles suitable for rental (isAvailable: true) in the system.
- GET /api/vehicles/:id : Returns all details of the vehicle with the specified ID.
- PUT /api/vehicles/:id/status : Updates the rental status of the vehicle (isAvailable: true/false). Can only be triggered with a valid x-internal-api-key header.

3.3. Rental Service

This is the service where the business logic is most concentrated. It allows users to rent vehicles between specific dates. It operates on Port: 3003.

- **Price Calculation Logic:** The millisecond difference between the rental start (startDate) and end (endDate) dates is calculated and converted into days. Accepting a minimum of 1 day, the number of days is multiplied by the daily price (dailyPrice) fetched dynamically from the vehicle service to calculate the totalPrice.
- **Workflow Consistency:** Immediately after the rental record is created, to prevent the relevant vehicle from being rented by someone else, an HTTP PUT request is sent to the vehicle-service to set the vehicle's isAvailable value to false.

API Endpoints:

- POST /api/rentals : Initiates the vehicle rental process (JWT Protected).
- GET /api/rentals/my-rentals : Lists all past and active rental records of the logged-in user.

4. INTER-MICROSERVICE COMMUNICATION PROTOCOL

In microservice architectures, communication between services is a critical issue. In this project, the **Synchronous HTTP Communication Model** was preferred, and the **Axios** library was used.

The scenario that occurs when a customer sends a vehicle rental request is as follows:

1. **Client (Customer)** sends a request to the rental-service with vehicleId, startDate, and endDate parameters. A JWT Token is added to the header of the request.
2. rental-service verifies the JWT token and extracts the user's userId information.
3. rental-service sends a synchronous **HTTP GET** request to vehicle-service (http://vehicle-service:3002/api/vehicles/<id>) to query the current status of the vehicle in the database.
4. Based on the response, it is checked whether the vehicle exists and if isAvailable === true. If it is not available, the rental is canceled, and a 400 Bad Request is returned.

5. If available, the total amount is calculated, and the rental record is saved to `rental_db`.
6. `rental-service` sends an **HTTP PUT** request to `vehicle-service` (`http://vehicle-service:3002/api/vehicles/<id>/status`) to change the vehicle's status to `isAvailable: false`. While sending this request, to prevent unauthorized persons from changing the vehicle status, the `INTERNAL_API_KEY` defined in the system environment variables is sent via the `x-internal-api-key` header.

This model ensures full integration and data consistency between services.

5. INFRASTRUCTURE AND DEVOPS PROCESSES (DOCKER & JENKINS PIPELINE)

5.1. Docker and Multi-Container Orchestration

All services and dependencies are packaged with Docker so they can run independent of the platform. The `docker-compose.yml` file in the root directory of the project is configured to bring up the entire orchestration with a single command.

- **Mongo Service:** A single MongoDB image (`mongo:7`) is deployed. `user_db`, `vehicle_db`, and `rental_db` databases are kept on this single server as logically isolated schemas. Thanks to the `mongo_data` volume, data is not lost even if the container is deleted (Data Persistence).
- **Port Configuration:** * MongoDB: 27017
 - User Service: 3001
 - Vehicle Service: 3002
 - Rental Service: 3003
 - Jenkins: 8080 and 50000 (For Agent communication)

5.2. Jenkins CI/CD Pipeline As Code

To automate the processes, a custom Jenkins image is defined with `Dockerfile.jenkins`. `docker.io` and `docker-compose` are installed in this image so that Jenkins can control the docker engine on the host machine from within itself.

With this pipeline, when the developer pushes code to Git, Jenkins detects the change, rebuilds the images, and automatically deploys the new version on docker with a Zero Downtime target.

6. OBSERVABILITY AND MONITORING STACK

Monitoring distributed architecture in microservice systems and identifying which service caused an error (root-cause analysis) is much more difficult compared to monolithic applications. To solve this problem, two major monitoring tools have been integrated into this project.

6.1. Datadog APM (Application Performance Monitoring) & Log Management

Datadog's APM library is integrated into the entry point of each microservice. Thanks to this library, HTTP requests received by the services, database queries, and inter-service Axios calls are automatically monitored (distributed tracing).

The `datadog-agent` service defined on `docker-compose.yml` listens to the Docker socket to:

- Collect CPU, Memory, and Network usage of containers.
- Automatically capture console logs produced by services.
- Transfer inter-service call delays (Latency), error rates, and throughput values to the Datadog cloud dashboard.

6.2. Prometheus Metrics Scrape

Prometheus configuration (`prometheus.yml`) has been added to the project to monitor system resources and service health using open standards. Prometheus regularly scans the defined targets and collects time-series metric data such as system uptime, request counts, and memory consumption.

7. API TEST SCENARIOS (POSTMAN)

The `CarRental-Postman.json` collection prepared for end-to-end verification of the system simulates a real rental scenario. The test steps and their validation logic are as follows:

1. **User Registration (Register - POST):**
 - URL: `http://localhost:3001/api/auth/register`
 - Body: Registration is performed with `admin@test.com` email and `admin` role.
2. **User Login (Login - POST):**
 - URL: `http://localhost:3001/api/auth/login`
 - Postman Test Script: Automatically captures the returned JWT Token upon successful login and writes it to the global `jwt_token` variable.
3. **Adding a New Vehicle (Vehicle Add - POST):**
 - URL: `http://localhost:3002/api/vehicles`
 - Header: Authorization: Bearer `{{jwt_token}}`
 - Body: Renault Megane (Plate: 34ABC123, Daily: 750 TL) data is sent.
 - Postman Test Script: Captures the unique `_id` value of the newly created vehicle and saves it to the global `vehicle_id` variable.
4. **Listing Available Vehicles (GET):**
 - URL: `http://localhost:3002/api/vehicles/available`
 - Lists the vehicles available for rent in the system (The vehicle we added is listed here).
5. **Vehicle Rental (Rental - POST):**
 - URL: `http://localhost:3003/api/rentals`
 - Header: Authorization: Bearer `{{jwt_token}}`
 - Body: Rental is initiated with the dynamic `{{vehicle_id}}` parameter and a date range of `2026-06-01` to `2026-06-05` (4 days).
 - Result: The system calculates a total price of $4 * 750 = 3000$ TL for 4 days and creates the rental record. It sends a request to `vehicle-service` and puts the vehicle in `rented` status.
6. **User Rental History (GET):**
 - URL: `http://localhost:3003/api/rentals/my-rentals`
 - Successfully displays the user's 3000 TL rental record.
7. **Status Check (GET):**
 - URL: `http://localhost:3002/api/vehicles/available`

- When called again, it is verified that the rented vehicle is no longer visible in the list. This proves that synchronous communication between services works flawlessly.

8. ACADEMIC EVALUATION AND CONCLUSION

The work carried out within the scope of this project has solved many architectural and operational problems encountered in modern cloud architectures:

- **Distributed Data Management:** With the "Database-per-Service" approach, the dependency of services on each other at the database level is prevented, and data security and microservice independence are maximized.
- **Secure Inter-Service Communication:** By using the `x-internal-api-key` check for internal communication, it is guaranteed that sensitive data updates (such as changing the vehicle status) are only performed by authorized services.
- **Cloud Integration Suitability:** Containerizing all services with Docker and automating them with Jenkins makes it extremely easy to migrate the application to modern cloud platforms like AWS, Azure, Google Cloud, or Kubernetes (K8s).
- **Observability Standards:** Thanks to the APM and log collection infrastructure, debugging time in distributed systems is minimized, and a monitoring mechanism at production environment standards has been established.

In summary; this study is a comprehensive academic project that successfully brings together software engineering design, system security, DevOps practices, and observability methodologies based on a car rental scenario.